

08

Geometric Transformations

Scaling, Rotation, Shear, Orthogonal Projection, Reflection, Affine Transformation (Including Translation)...

This chapter covers common geometric transformations, which are widely used in fields such as computer graphics and robotics. Beyond that, nearly every subsequent chapter in this book relies on the geometric transformations introduced here.

Linear Algebra Made Easy, Volume 1, already introduced various geometric operations here and there in different contexts; this chapter now systematically covers the common geometric operations, along with the various linear algebra tools underlying them.

This chapter is actually a great opportunity to review and reinforce earlier material, since it draws on many important linear algebra concepts covered previously — matrix multiplication, inverse matrices, determinants, linear transformations, bases, and more.

This chapter also serves as a bridge to what comes next, since its content is used extensively in the various matrix decompositions introduced later in the book (such as QR decomposition, eigendecomposition, Cholesky decomposition, LDL decomposition, singular value decomposition, and more); these geometric transformations also come up frequently in data analysis and machine learning applications.

For each geometric transformation in this chapter, we especially recommend examining it from the following angles:

- ▶ Geometric view of matrix multiplication (especially how the matrix transforms the zero vector $\mathbf{0}$, the unit vectors $\mathbf{e}_1$, $\mathbf{e}_2$, and the all-ones column vector $\mathbf{1}$).
- ▶ Look at the determinant (whether its absolute value is greater than 1; whether it's positive, negative, or zero); and the determinants of special matrices (diagonal matrices, upper triangular matrices, lower triangular matrices, ...).
- ▶ Use the determinant to see how the geometric transformation changes area or volume.
- ▶ Look at the inverse matrix (first checking whether it exists) to understand the inverse transformation; and the inverses of special matrices (diagonal matrices, upper triangular matrices, lower triangular matrices, orthogonal matrices, ...).
- ▶ From the linear-combination perspective, see how the parallelogram grid or parallelepiped shape changes, and whether the transformation causes a reduction in dimension.
- ▶ From the basis perspective, see whether the transformation matrix's columns form a basis, and if so, what kind of basis (standard orthonormal basis, orthonormal basis, orthogonal basis, or non-orthogonal basis).
- ▶ The properties of the transformation matrix (whether it's symmetric, diagonal, orthogonal, or invertible — i.e., whether its determinant is zero, and whether matrix multiplication is commutative, ...).

8.1 Geometric Transformations and Matrix Multiplication



Core skills you'll master in this section:

- ▶ Linear mapping: through the matrix multiplication $Ax = y$, understand how the input vector x is mapped to the output vector y .
- ▶ Under a linear transformation, the original unit-square grid is transformed into a parallelogram grid; these parallelograms are arranged in parallel and evenly spaced, and the origin's position stays unchanged.
- ▶ The geometric meaning of special column vectors: the zero vector, the unit vectors, and the all-ones column vector.
- ▶ Reviewing the transpose of a matrix product: $(AB)^T = B^T A^T$.
- ▶ Scaling: a diagonal matrix.
- ▶ Rotation: an orthogonal linear transformation that preserves length and angle; an orthogonal matrix with determinant 1.
- ▶ Shear: an upper/lower triangular matrix whose main-diagonal entries are all 1, with determinant 1.
- ▶ Orthogonal projection: reduces dimension, has determinant 0, and is a non-invertible matrix.
- ▶ Reflection: an orthogonal matrix with determinant -1 .

This chapter will introduce scaling, rotation, shear, orthogonal projection, and reflection one by one; this section starts by discussing these geometric transformations from the perspective of matrix multiplication. The content here is mainly a review of core points from the first seven chapters of the book, so it's relatively simple – this section will not go into detail on these individual geometric transformations.

Reviewing the Matrix Multiplication $Ax = y$

As introduced earlier in the book, and as shown in Figure 1, matrix multiplication is equivalent to using the matrix A to apply a linear mapping to the column vector x .

$$A_{m \times p} x_{p \times 1} = y_{m \times 1}$$

Looking at the matrix shapes, in $(m \times p) @ (p \times 1)$ the (p) in the middle gets “cancelled out,” so the result of the matrix multiplication has shape $(m \times 1)$.

The p -dimensional column vector x is like an input vector, representing some initial state; x is a point in $\mathbb{R}^p$ space.

The m -dimensional column vector $\mathbf{y}$ is like an output vector, representing the result after the transformation; $\mathbf{y}$ is a point in $\mathbb{R}^m$ space.

From the linear-combination perspective, the column vector $\mathbf{y}$ is a linear combination of the column vectors of matrix $\mathbf{A}$.

⚠ Note that the matrix $\mathbf{A}$ is a wide (flat) matrix, so $\mathbf{A}$'s column vectors clearly do not form a basis.

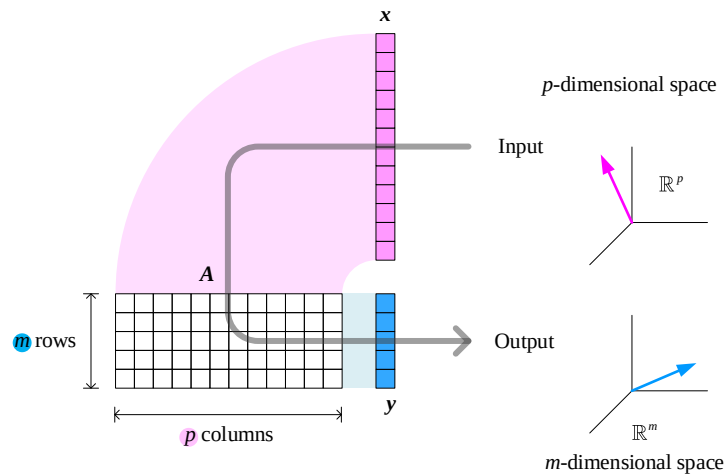


Figure 1. Matrix Multiplication $\mathbf{Ax} = \mathbf{y}$, Input and Output

2 × 2 Square Matrix

Given a 2 × 2 matrix $\mathbf{A}$

$$\mathbf{A} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}$$

Using the third perspective on matrix multiplication, this expands to $\mathbf{Ax} = \mathbf{y}$

$$\mathbf{Ax} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} a_{1,1} \\ a_{2,1} \end{bmatrix} + x_2 \begin{bmatrix} a_{1,2} \\ a_{2,2} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \mathbf{y}$$

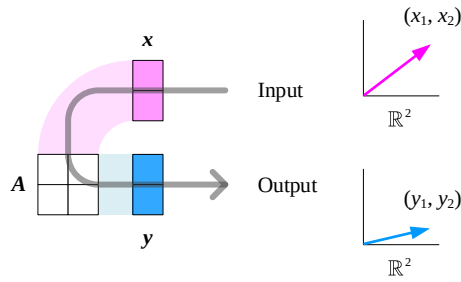
$\mathbf{a}_1 \qquad \mathbf{a}_2$

Looking at the matrix shapes, in $(2 \times 2) @ (2 \times 1)$ the (2) in the middle gets “cancelled out,” so the result of the matrix multiplication has shape (2×1) .

As shown in Figure 2, for the matrix multiplication $\mathbf{Ax} = \mathbf{y}$, the matrix $\mathbf{A}$ has shape 2×2 .

The input vector $\mathbf{x}$ is a 2×1 vector, corresponding to a point in the Cartesian plane $\mathbb{R}^2$ with coordinates (x_1, x_2) ; equivalently, it can be viewed as a vector in the plane starting at the origin $(0,0)$ and ending at (x_1, x_2) .

The output vector $\mathbf{y}$ is also a 2×1 vector, and is likewise a point in $\mathbb{R}^2$ with coordinates (y_1, y_2) ; it is also a vector in the Cartesian plane $\mathbb{R}^2$ starting at the origin $(0,0)$ and ending at (y_1, y_2) . The column vector $\mathbf{y}$ is a linear combination of $\mathbf{a}_1$ and $\mathbf{a}_2$.

Figure 2. $Ax = y$, Matrix A is 2×2

Parallelogram

Now let's look at the matrix multiplication for a few special column vectors.

As shown in Figure 3, let's focus on the four column vectors corresponding to the vertices of the unit square (light yellow) in the left panel: the zero vector 0 , the unit vectors e_1 , e_2 , and the all-ones column vector 1 .

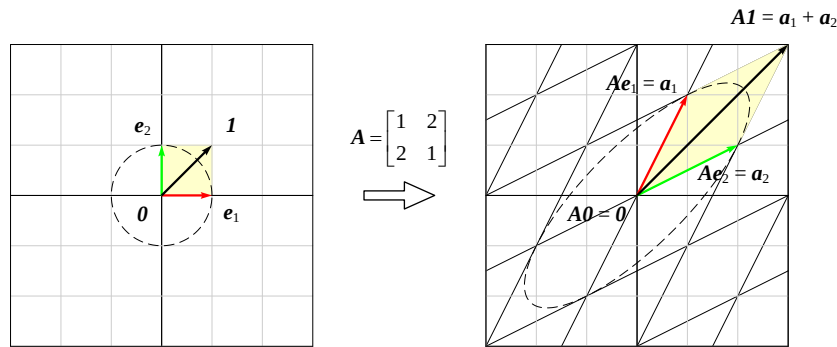


Figure 3. Four Special Column Vectors in the Plane

As shown in the right panel of Figure 3, after the linear mapping $Ax = y$, the unit square becomes a parallelogram.

The four vertices of the parallelogram are:

- ▶ The zero vector 0 , corresponding to the matrix multiplication $A0$
- ▶ A 's first column vector a_1 , corresponding to the matrix multiplication Ae_1
- ▶ A 's second column vector a_2 , corresponding to the matrix multiplication Ae_2
- ▶ A 's sum of the first two column vectors, $a_1 + a_2$, corresponding to the matrix multiplication $A1$

The unit square corresponds to an area of 1, matching the determinant of the identity matrix I . Viewed 180 degrees counterclockwise, the identity matrix I 's first column vector e_1 leads its second column vector e_2 .

Figure 3, the determinant of the square matrix A is -3 , meaning the parallelogram has area 3; the reason for the negative sign is that, viewed 180 degrees counterclockwise, A 's first column vector a_1 lags behind its second column vector a_2 .

Matrix Multiplication for the Four Special Column Vectors

Now let's look at these four matrix multiplications.

The zero vector $\mathbf{0}$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{0}$

$$\mathbf{A}\mathbf{0} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \mathbf{0}$$

We find that the result is still the zero vector $\mathbf{0}$, meaning the origin's position doesn't change! This is especially important for linear transformations!

The unit vector $\mathbf{e}_1$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{e}_1$

$$\mathbf{A}\mathbf{e}_1 = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{1,1} \\ a_{2,1} \end{bmatrix} = \mathbf{a}_1$$

The matrix multiplication above is equivalent to extracting $\mathbf{A}$'s first column vector $\mathbf{a}_1$. This corresponds to Figure 3, the red vector in the parallelogram on the right.

The unit vector $\mathbf{e}_2$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{e}_2$

$$\mathbf{A}\mathbf{e}_2 = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} a_{1,2} \\ a_{2,2} \end{bmatrix} = \mathbf{a}_2$$

The expression above is equivalent to extracting $\mathbf{A}$'s second column vector $\mathbf{a}_2$. This corresponds to Figure 3, the green vector in the parallelogram on the right.

The all-ones column vector $\mathbf{1}$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{1}$

$$\mathbf{A}\mathbf{1} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} a_{1,1} + a_{1,2} \\ a_{2,1} + a_{2,2} \end{bmatrix} = \mathbf{a}_1 + \mathbf{a}_2$$

The expression above is equivalent to computing the sum of $\mathbf{A}$'s column vectors, $\mathbf{a}_1 + \mathbf{a}_2$.

$\mathbf{a}_1 + \mathbf{a}_2$ corresponds to Figure 3, the black vector in the parallelogram on the right.

Also, notice another detail in Figure 3: the change in the grid.

Figure 3's left panel shows a grid of unit squares, representing the space spanned by the standard orthonormal basis $[\mathbf{e}_1, \mathbf{e}_2]$, corresponding to the linear combination

$$k_1\mathbf{e}_1 + k_2\mathbf{e}_2$$

Here, k_1 and k_2 can be any real numbers.

As shown in the right panel of Figure 3, after the linear transformation by the matrix $\mathbf{A}$, if $\mathbf{a}_1$ and $\mathbf{a}_2$ are linearly independent, the parallelogram grid corresponds to the space spanned by the basis $[\mathbf{a}_1, \mathbf{a}_2]$, corresponding to the linear combination

$$k_1\mathbf{a}_1 + k_2\mathbf{a}_2$$

Here, k_1 and k_2 can likewise be any real numbers.

Under a linear transformation, the original unit-square grid is transformed into a parallelogram grid; these parallelograms are arranged in parallel and evenly spaced, and the origin's position stays unchanged.

Being merely “parallel” isn't enough — we specifically emphasize “evenly spaced” too! As shown in Figure 4, the result of transforming the unit-square grid is still a parallel grid, but it isn't evenly spaced. So the two transformations in Figure 4 are not linear transformations.

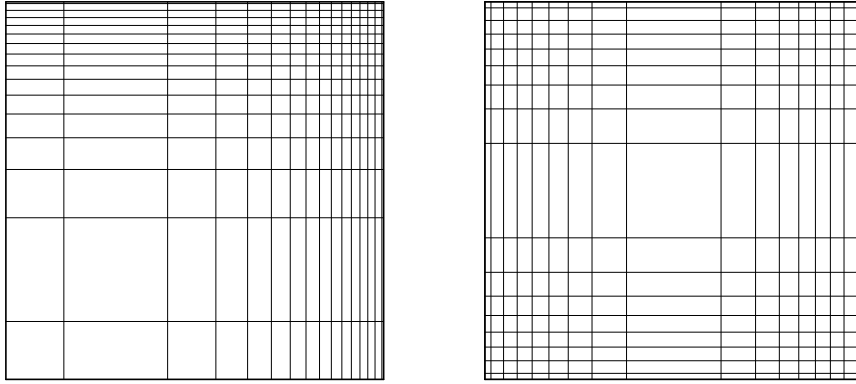


Figure 4. Non-Linear Transformations



You may also have noticed that in Figure 3's left panel, the unit circle becomes an ellipse. This is something we'll cover in depth later in the book; for now, this chapter will set it aside — just keep an impression of it for now.

Transpose of Matrix Multiplication

Taking the transpose of in the matrix multiplication gives

$$\begin{aligned} (\mathbf{Ax})^T &= \mathbf{x}^T \mathbf{A}^T = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \end{bmatrix} \\ &= x_1 \mathbf{a}_1^T + x_2 \mathbf{a}_2^T \\ &= x_1 \begin{bmatrix} a_{1,1} & a_{2,1} \end{bmatrix} + x_2 \begin{bmatrix} a_{1,2} & a_{2,2} \end{bmatrix} \\ &= \begin{bmatrix} y_1 & y_2 \end{bmatrix} = \mathbf{y}^T \end{aligned}$$

The expression above uses a 2D row vector to describe a point in the plane.

This expression helps us recall a property of matrix multiplication

$$(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$$

This tells us that the following matrix multiplication is equivalent to a sequence of successive geometric transformations

$$\mathbf{DCBAx}$$

The order in which these act on the column vector $\mathbf{x}$ goes from right to left, that is, $\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C} \rightarrow \mathbf{D}$.

Taking the transpose of gives

$$\mathbf{x}^T \mathbf{A}^T \mathbf{B}^T \mathbf{C}^T \mathbf{D}^T$$

In this case, the order of action on $\mathbf{x}^T$ goes from left to right.

Now let's briefly discuss the matrix multiplication operations corresponding to different geometric transformations.

Scaling

As shown in Figure 5, scaling in the plane is a linear transformation that stretches or compresses a planar geometric shape proportionally; the corresponding transformation matrix A is a diagonal matrix.

The unit-square grid becomes a square grid (as in Figure 5) or a rectangular grid.

➡ The scaling transformation is important – topics covered later in the book, such as eigendecomposition, singular value decomposition, and principal component analysis, all make use of it.

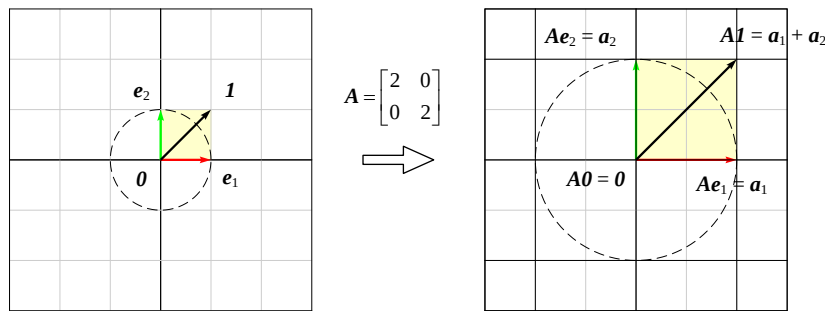


Figure 5. Matrix A is 2×2 , Scaling

Writing Figure 5's square matrix A as the column vectors $[a_1, a_2]$, specifically as follows

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

If a_1 and a_2 are linearly independent, the basis $[a_1, a_2]$ is an orthogonal basis, and the grid is a rectangle (a square being a special case of a rectangle).

Rotation

As shown in Figure 6, rotating the plane about the origin is a linear transformation that preserves length and angle; its corresponding square matrix is an orthogonal matrix with determinant 1.

Recall the properties of orthogonal matrices introduced earlier. If the matrix A is an orthogonal matrix, then A satisfies $AA^T = A^T A = I$.

Looking at the grid transformation in Figure 6, we can readily see that the axis-aligned unit-square grid (the standard orthonormal basis) becomes a rotated unit-square grid (an orthonormal basis).

➡ The rotation transformation is especially important – we'll make extensive use of it in topics such as spectral decomposition, singular value decomposition, and principal component analysis.

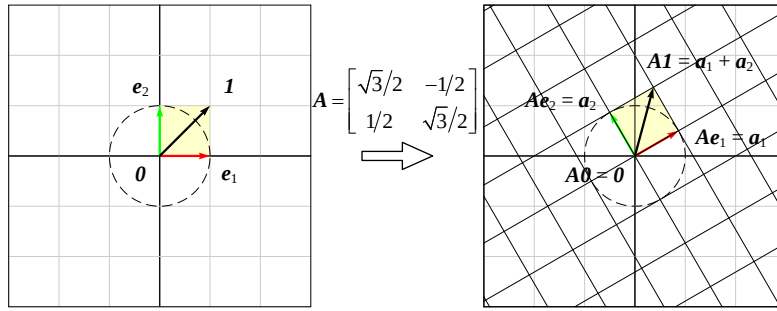
Figure 6. Matrix A is 2×2 , Rotation (About the Origin)

Figure 6, the matrix A 's column vectors a_1 and a_2 are both unit vectors and orthogonal to each other.

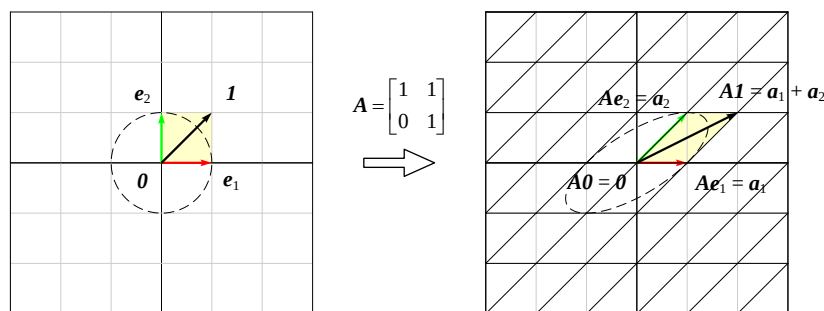
Shear

As shown in Figure 7, shear in the plane is a linear transformation that tilts and deforms a shape along a certain direction; the matrix corresponding to this linear transformation is a triangular matrix (upper or lower triangular, with all main-diagonal entries equal to 1).

The square matrix corresponding to a shear transformation has determinant 1, meaning area is preserved.

➡ Interestingly, in Figure 7, the unit circle also becomes an ellipse – this is a topic we'll discuss later in the book in connection with LDL decomposition.

The matrix A 's column vector a_1 is a unit vector pointing in the positive direction of the horizontal axis. To be precise, Figure 7 shows a horizontal shear.

Figure 7. Matrix A is 2×2 , Horizontal Shear

Orthogonal Projection

As shown in Figure 8, orthogonal projection in the plane is a linear transformation that projects points perpendicularly onto some subspace, causing the shape to lose a dimension.

⚠ Note that the square matrix corresponding to an orthogonal projection has determinant 0, which means the orthogonal projection matrix is not invertible!

This also means the square matrix's column vectors are linearly dependent and cannot form a basis! For example, in Figure 8, the matrix A 's column vectors a_1 and a_2 satisfy $a_1 + a_2 = 0$.



Orthogonal projection is closely related to the scalar projection and vector projection introduced earlier in the book; it's also closely related to Gram–Schmidt orthogonalization, QR decomposition, and least squares — these are core topics we'll discuss in the next chapter.

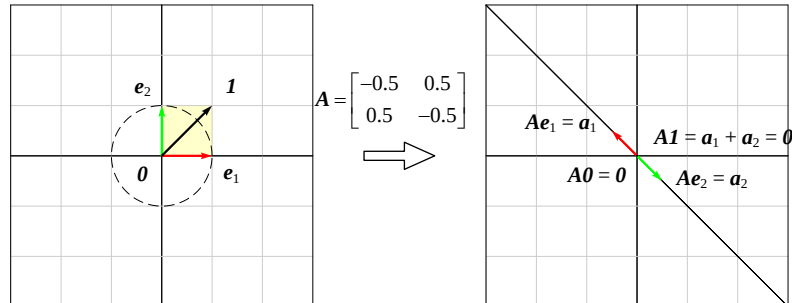


Figure 8. Matrix A is 2×2 , Orthogonal Projection

Reflection

As shown in Figure 9, reflection in the plane is a linear transformation that flips shapes symmetrically about some line through the origin; the square matrix corresponding to this linear transformation is also an orthogonal matrix, with determinant -1 .

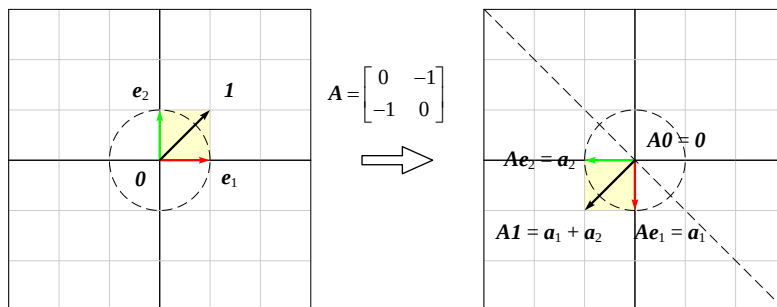


Figure 9. Matrix A is 2×2 , Reflection

Figure 9, the matrix A 's column vectors a_1 and a_2 are also both unit vectors and orthogonal. However, this square matrix differs from the rotation matrix in Figure 6.

Visualization

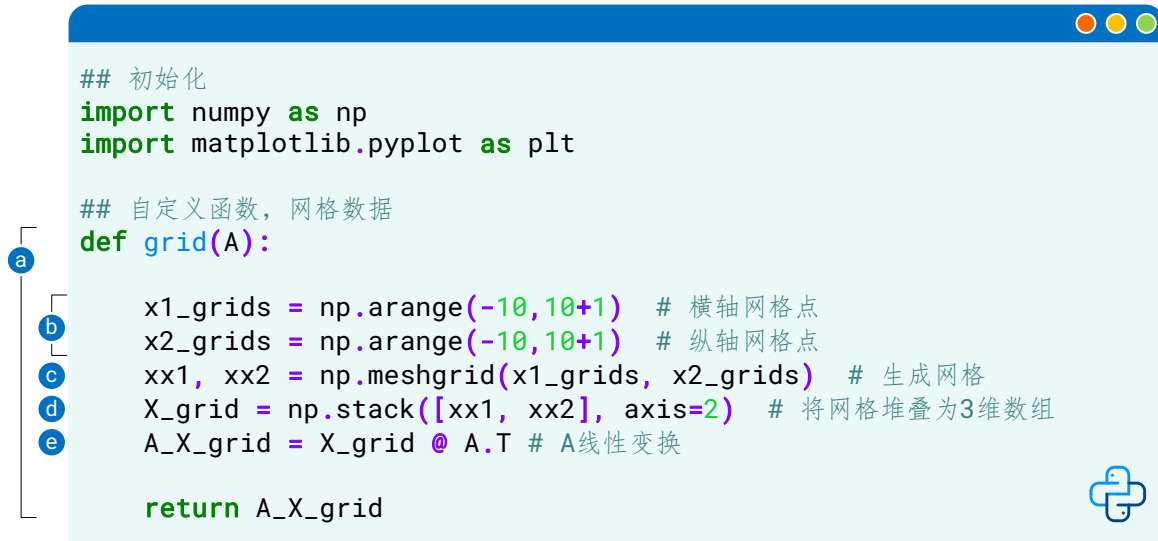
The code file LA_08_01_01.ipynb produces the right panel of Figure 3.

Let's first go over the custom functions used in it.

Code 1 defines a function called `grid`, which takes one parameter A , a matrix describing a 2D linear transformation.

- a** is the body of the custom function.
- b** Uses `numpy.arange()` to create integer grid points along the horizontal and vertical axes.
- c** Uses the `numpy.meshgrid()` function to expand the two 1D arrays above into a 2D grid; `xx1` holds all the horizontal coordinates, and `xx2` holds all the vertical coordinates.

- d Uses `numpy.stack()` to stack the two grids together, forming a 3D array that makes the subsequent matrix operations more convenient.
- e Applies the linear transformation to every point in this grid; `@` is the matrix multiplication operator, and `A.T` denotes the transpose of `A`, used to match the dimensions. After the transformation, the position of every point on the grid changes.

Code 1. Custom Function: Grid Data |  LA_08_01_01.ipynb


```

## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 自定义函数, 网格数据
def grid(A):
    x1_grids = np.arange(-10, 10+1) # 横轴网格点
    x2_grids = np.arange(-10, 10+1) # 纵轴网格点
    xx1, xx2 = np.meshgrid(x1_grids, x2_grids) # 生成网格
    X_grid = np.stack([xx1, xx2], axis=2) # 将网格堆叠为3维数组
    A_X_grid = X_grid @ A.T # A线性变换

    return A_X_grid

```

Code 2 is a custom function that creates the unit-circle data.

- a Defines a function called `circle`, which likewise takes a transformation matrix `A`; its job is to generate unit-circle data and apply the linear transformation. Finally, it uses `return` to return the transformed data for use by other functions.
- b Uses `numpy.linspace()` to create an array representing angles from 0 to a full 360° turn (expressed in radians, i.e., 2π), with 721 points in total.
- c With the default radius of 1 (the unit circle), computes the cosine (horizontal coordinate) and sine (vertical coordinate) for each angle.
- d Uses `numpy.column_stack()` to combine the horizontal and vertical coordinates into a 2D array, where each row is a point on the unit circle.
- e Applies the linear transformation to every point on the unit circle to obtain the data for the transformed outline; in this example, the result is a rotated ellipse.

Code 2. Custom Function: Unit Circle Data |  LA_08_01_01.ipynb

```

## 自定义函数，单位圆数据
def circle(A):
    theta_array = np.linspace(0, 2*np.pi, 721)
    # 极坐标到直角坐标
    x1_array = np.cos(theta_array)
    x2_array = np.sin(theta_array)
    unit_circle = np.column_stack([x1_array, x2_array])

    A_unit_circle = unit_circle @ A.T # A线性变换

    return A_unit_circle

```

Code 3 is a custom function that creates the unit-square data and applies the linear transformation. The code is fairly simple, so we'll leave the analysis to you. What's worth noting is that in **b**, the four row vectors represent the four vertices of the unit square.

Code 3. Custom Function: Unit Square Data |  LA_08_01_01.ipynb

```

## 自定义函数，单位正方形数据
def unit_square(A):
    unit_square = np.array([[0, 0],
                             [1, 0],
                             [1, 1],
                             [0, 1]])

    A_unit_square = unit_square @ A.T # A线性变换

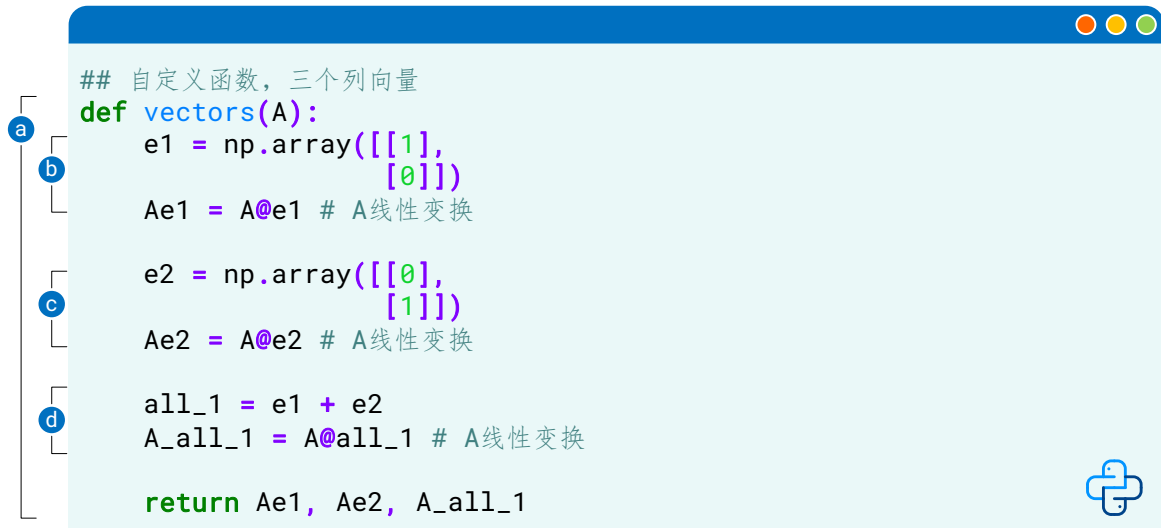
    return A_unit_square

```

Code 4 is a custom function that creates the vector data (the unit vectors and the all-ones vector).

- a** Defines a function called `vectors`, whose job is to generate the unit vectors $\mathbf{e}_1$ and $\mathbf{e}_2$, and the all-ones column vector $\mathbf{1}$, and to examine the effect of the linear transformation on them.
- b** Defines the unit vector $\mathbf{e}_1$ pointing in the positive horizontal-axis direction. Then applies the linear transformation to this vector.
- c** Defines the unit vector $\mathbf{e}_2$ pointing in the positive vertical-axis direction. Then applies the same linear transformation to this vector.
- d** Adds $\mathbf{e}_1$ and $\mathbf{e}_2$ together to get the all-ones column vector, then applies the linear transformation to it.

Code 4. Custom Function: Vector Data |  LA_08_01_01.ipynb



```

## 自定义函数，三个列向量
def vectors(A):
    e1 = np.array([[1],
                   [0]])
    Ae1 = A@e1 # A线性变换

    e2 = np.array([[0],
                   [1]])
    Ae2 = A@e2 # A线性变换

    all_1 = e1 + e2
    A_all_1 = A@all_1 # A线性变换

    return Ae1, Ae2, A_all_1

```

Code 5 is the custom visualization function.

- a Defines a function called visualize, used to plot all the transformed shapes from above.
- b Calls the custom functions defined earlier to generate the transformed grid, unit square, unit circle, and the three vectors' data, respectively.
- c First plots the transformed grid row by row to draw the horizontal grid lines, then plots it column by column to draw the vertical grid lines.
- d Uses `quiver()` to draw the three arrows. This function has been covered many times before, so please review its parameters on your own.
- e Uses `plot()` to draw the result of applying the linear transformation to the unit circle.
- f Uses `fill()` to draw the result of applying the transformation to the unit square, filled with a light yellow background color.

Code 5. Custom Visualization Function |  LA_08_01_01.ipynb

```

## 定义可视化函数
def visualize(A):
    fig, ax = plt.subplots()

    # 调用自定义函数创建数据
    A_X_grid = grid(A)
    A_unit_square = unit_square(A)
    A_unit_circle = circle(A)
    Ae1, Ae2, A_all_1 = vectors(A)

    # 绘制变换后的网格
    for i in range(A_X_grid.shape[0]):
        plt.plot(A_X_grid[i, :, 0], A_X_grid[i, :, 1],
                 'k-', linewidth=0.5) # 绘制网格行
    for j in range(A_X_grid.shape[1]):
        plt.plot(A_X_grid[:, j, 0], A_X_grid[:, j, 1],
                 'k-', linewidth=0.5) # 绘制网格列

    # 绘制变换后的列向量
    plt.quiver(0, 0, Ae1[0], Ae1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[1, 0, 0], zorder = 1e5)
    plt.quiver(0, 0, Ae2[0], Ae2[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 1, 0], zorder = 1e5)
    plt.quiver(0, 0, A_all_1[0], A_all_1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 0, 0], zorder = 1e5)

    # 绘制变换后单位圆
    plt.plot(A_unit_circle[:,0], A_unit_circle[:,1],
             c = 'k', ls = '--')

    # 绘制变换后单位正方形
    plt.fill(A_unit_square[:,0], A_unit_square[:,1],
             c = '#FFFFCC', ls = '--', zorder = 1)

    # 装饰
    plt.axvline(x=0, color='k', zorder=0)
    plt.axhline(y=0, color='k', zorder=0)
    ax.tick_params(axis='both', which='both', length=0,
                  labelbottom=False, labelleft=False)
    ax.set_aspect(1)
    lim = 3; ax.set_xlim([-lim, lim]); ax.set_ylim([-lim, lim])
    plt.xticks(np.arange(-lim, lim))
    plt.yticks(np.arange(-lim, lim))
    ax.grid(linestyle='--', linewidth=0.15, color=[0.8, 0.8, 0.8])

```

3 × 3 Square Matrix

Finally, let's extend the linear transformation to 3D space. Given the 3 × 3 matrix A

$$\mathbf{A} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

The 3×3 matrix $\mathbf{A}$ maps the 3D vector $\mathbf{x}$ to the 3D vector $\mathbf{y}$.

As shown in Figure 10, for the matrix multiplication $\mathbf{Ax} = \mathbf{y}$, when the matrix $\mathbf{A}$ has shape 3×3 (3 rows, 3 columns), the input vector $\mathbf{x}$ is a 3×1 vector, and $\mathbf{x}$ represents a coordinate point in $\mathbb{R}^3$ space.

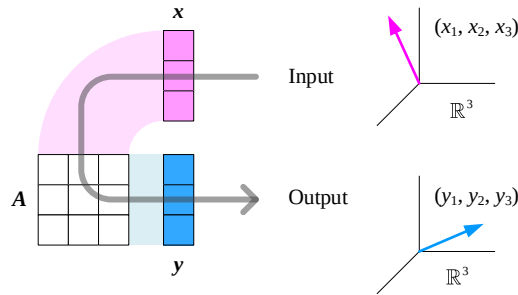


Figure 10. Matrix $\mathbf{A}$ is 3×3

The output vector $\mathbf{y}$ is also a 3×1 vector; $\mathbf{y}$ is also a coordinate point in 3D space $\mathbb{R}^3$.

Looking at the matrix shapes, in $(3 \times 3) @ (3 \times 1)$ the (3) in the middle gets “cancelled out,” so the result of the matrix multiplication has shape (3×1) .

Similarly, let's look at a few special matrix multiplications, shown specifically in Figure 11.

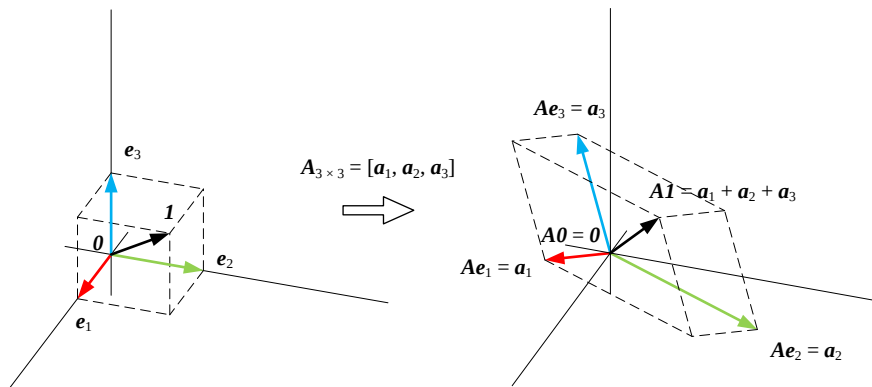


Figure 11. Several Special Column Vectors in 3D Space

The zero vector $\mathbf{0}$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{0}$

$$\mathbf{A}\mathbf{0} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \mathbf{0}$$

Likewise, in 3D space, the linear transformation leaves the origin's position unchanged.

The unit vector $\mathbf{e}_1$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{e}_1$

$$\mathbf{A}\mathbf{e}_1 = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{1,1} \\ a_{2,1} \\ a_{3,1} \end{bmatrix} = \mathbf{a}_1$$

The matrix multiplication above extracts the square matrix $\mathbf{A}$'s first column.

The unit vector $\mathbf{e}_2$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{e}_2$

$$\mathbf{A}\mathbf{e}_2 = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{1,2} \\ a_{2,2} \\ a_{3,2} \end{bmatrix} = \mathbf{a}_2$$

The expression above extracts $\mathbf{A}$'s second column.

The unit vector $\mathbf{e}_3$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{e}_3$

$$\mathbf{A}\mathbf{e}_3 = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} a_{1,3} \\ a_{2,3} \\ a_{3,3} \end{bmatrix} = \mathbf{a}_3$$

The expression above extracts $\mathbf{A}$'s third column.

The all-ones column vector $\mathbf{1}$, under the linear transformation, corresponds to the matrix multiplication $\mathbf{A}\mathbf{1}$

$$\mathbf{A}\mathbf{1} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} a_{1,1} + a_{1,2} + a_{1,3} \\ a_{2,1} + a_{2,2} + a_{2,3} \\ a_{3,1} + a_{3,2} + a_{3,3} \end{bmatrix} = \mathbf{a}_1 + \mathbf{a}_2 + \mathbf{a}_3$$

The expression above sums $\mathbf{A}$'s three column vectors.

? A matrix $\mathbf{A}$ multiplied by a (shape-matching) all-ones column vector, i.e., $\mathbf{A}\mathbf{1}$, gives the sum of the row vectors. Now think about this: the transpose of the all-ones column vector, $\mathbf{1}^T$ (shape-matching), multiplied on the left of a matrix $\mathbf{A}$, i.e., $\mathbf{1}^T\mathbf{A}$ — what result does that give? And what does the matrix multiplication $\mathbf{1}^T\mathbf{A}\mathbf{1}$ give? $\mathbf{1}^T\mathbf{A}\mathbf{1}$ Are the two all-ones vectors the same shape? When are they the same?

As shown in Figure 11, we find that the unit cube becomes a parallelepiped.

Figure 11's left panel represents the space spanned by the standard orthonormal basis $[\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3]$, corresponding to the linear combination

$$k_1\mathbf{e}_1 + k_2\mathbf{e}_2 + k_3\mathbf{e}_3$$

Here, k_1 , k_2 , and k_3 can be any real numbers.

Figure 11's right panel shows that after the linear transformation by the matrix $\mathbf{A}$, if $\mathbf{a}_1$, $\mathbf{a}_2$, and $\mathbf{a}_3$ are linearly independent, the parallelepiped corresponds to the space spanned by the basis $[\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$, corresponding to the linear combination

$$k_1\mathbf{a}_1 + k_2\mathbf{a}_2 + k_3\mathbf{a}_3$$

Here, k_1 , k_2 , and k_3 can likewise be any real numbers.

To echo what we said earlier in this section, under a linear transformation, the original unit-cube grid is transformed into a parallelepiped grid; these parallelepipeds are arranged in parallel and evenly spaced, and the origin's position stays unchanged.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Use matrix multiplication to compute how each of the following square matrices transforms the zero vector $\mathbf{0}$, the unit vectors $\mathbf{e}_1$ and $\mathbf{e}_2$, and the all-ones column vector $\mathbf{1}$.

▶ $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$

▶ $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

▶ $\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$

▶ $\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$

▶ $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$

▶ $\begin{bmatrix} 1/2 & 0 \\ 0 & 1/2 \end{bmatrix}$

▶ $\begin{bmatrix} 1/2 & 0 \\ 0 & 2 \end{bmatrix}$

▶ $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$

▶ $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}$

▶ $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$

▶ $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Q2. Describe in words, roughly, what geometric transformation each square matrix in Q1 performs. We suggest first making a judgment based on your Q1 results, then using this section's accompanying code to plug in each matrix, compute, and visualize it to check your judgment.

Q3. Analyze which matrices in Q1 have linearly dependent column vectors.

Q4. Analyze which matrices in Q1 have column vectors that form a basis, identify the type of basis, and explain why.

Q5. Compute the length (magnitude, norm, L^2 norm) of each column vector of the square matrices in Q1, and the angle between the column vectors (using the inner product).

Q6. On graph paper, draw the parallelogram formed by the column vectors of each square matrix in Q1.

Q7. Compute the determinant of each square matrix in Q1; think about what properties each matrix has and what the most convenient way to compute it is; and explain why the determinant is positive, negative, or zero.

Q8. Explain whether each square matrix in Q1 has an inverse; if it does, compute the inverse.